

CSC 830 - Advanced Artificial Intelligence

Lecture 03: Knowledge Representation and Reasoning - Part II

Prof. Nwojo Agwu Nnanna
nagwu@nileuniversity.edu.ng

March 24, 2025

Nile University of Nigeria, Abuja

Table of contents

1. First-order (Predicate) Logic
2. Inference in First-Order Logic
3. Unification of two clauses

First-order (Predicate) Logic

First-order (Predicate) Logic FOL

- To solve the limitations in the propositional calculus, you need to analyze propositions into predicates and arguments, and deal explicitly with quantification. Predicate Logic provides formalism for performing this analysis of prepositions and additional methods for reasoning with quantified expressions.
- For example, instead of letting a single propositional symbol, P , denote the entire sentence “it rained on Tuesday”, we can create a predicate `weather` that describes a relationship between a date and weather: `weather(tuesday, rain)`

- Consider the set $A = \{x | P(x)\}$. An element a in A is an object for which $P(x)$ is true. $P(x)$ is called a **predicate** or **propositional function**; each value of x leads to a proposition $P(x)$ that is either true or false.]item The propositional logic is not powerful enough to represent all types of assertions that are used in computer science and mathematics, or to express certain types of relationship between propositions such as equivalence.
- For example, the assertion " **x is greater than 1**", where x is a variable, is not a proposition because you can not tell whether it is true or false unless you know the value of x . Thus the propositional logic cannot deal with such sentences. However, such assertions appear quite often in mathematics and computer science and we want to do inferencing on those assertions.

Predicate

- To cope with deficiencies of propositional logic we introduce two new features: predicates and quantifiers. A predicate is a verb phrase template that describes a property of objects, or a relationship among objects represented by the variables.
- For example, the sentences "The car Yahya is driving is blue", "The sky is blue", and "The cover of this book is blue" come from the template "is blue" by placing an appropriate noun/noun phrase in front of it. The phrase "is blue" is a predicate and it describes the property of being blue.
- Predicates are often given a name. For example any of "is-blue", "Blue" or "B" can be used to represent the predicate "is blue" among others. If we adopt B as the name for the predicate "is-blue", sentences that assert an object is blue can be represented as " $B(x)$ ", where x represents an arbitrary object. $B(x)$ reads as " x is blue".

- Similarly the sentences "Emeka gives the book to Chioma", "Audu gives a loaf of bread to Fatima", and "Jane gives a lecture to Linda" are obtained by substituting an appropriate object for variables x , y , and z in the sentence " x gives y to z ". The template ". . . gives . . . to . . ." is a predicate and it describes a relationship among three objects. This predicate can be represented by $\text{Give}(x, y, z)$ or $G(x, y, z)$, for example.

- **Note:** The sentence "Emeka gives the book to Chioma" can also be represented by another predicate such as "gives the book to". Thus if we use $B(x, y)$ to denote this predicate, "Emeka gives the book to Chioma" becomes $B(\text{Emeka}, \text{Chioma})$. In that case, the other sentences, "Audu gives a loaf of bread to Fatima", and "Jane gives a lecture to Linda", must be expressed with other predicates.

First-Order (Predicate) Logic

- First-order logic is used to model the world (the universe of discourse) in terms of
 - **objects** which are things with individual identities e.g., individual students, lecturers, companies, cars . . .
 - **properties** of objects that distinguish them from other objects e.g., mortal, blue, oval, even, large, . . .
 - **classes** of objects (often defined by properties) e.g., human, mammal, machine, . . .
 - **relations** that hold among objects e.g., brother of, bigger than, outside, part of, has color, occurs after, owns, a member of, . . .
 - **functions** which are a subset of the relations in which there is only one “value” for any given “input”. e.g., father of, best friend, second half, one more than . . .

- **Predicates:** $P(x_1, \dots, x_n)$
 - P : predicate name; $(x_1, \dots, x_n)$: argument list
 - A special function with range = $\{T, F\}$;
 - Examples: $\text{human}(x)$, /* x is a human */ $\text{father}(x, y)$ /* x is the father of y */
 - When all arguments of a predicate are assigned values (said to be instantiated), the predicate becomes either true or false, i.e., it becomes a proposition. Ex. $\text{father}(\text{Maduka}, \text{Ngozi})$
 - A predicate, like a membership function, defines a set (or a class) of objects
- **Terms** (arguments of predicates must be terms)
 - **Constants** are terms (e.g., Fatima, a, Z, “red”, etc.)
 - **Variables** are terms (e.g., x, y, z , etc.), a variable is instantiated when it is assigned a constant as its value
 - **Functions** of terms are terms (e.g., $f(x, y, z)$, $f(x, g(a))$, etc.)
 - A term is called a ground term if it does not involve variables
 - **Predicates**, though special functions, are not terms in FOL

- **Universal quantification $\forall$ (or forall)**

- $(\forall x)P(x)$ means that P holds for all values of x in the domain associated with that variable.
- For example:
 - $(\forall x)\text{dolphin}(x) \Rightarrow \text{mammal}(x)$
 - $(\forall x)\text{human}(x) \Rightarrow \text{mortal}(x)$
- Universal quantifiers often used with "implication ($\Rightarrow$)" to form "rules" about properties of a class $(\forall x)\text{student}(x) \Rightarrow \text{smart}(x)$ (All students are smart)
- Often associated with English words "all", "everyone", "always", etc.
- You rarely use universal quantification to make blanket statements about every individual in the world (because such statement is hardly true) $(\forall x)\text{student}(x) \wedge \text{smart}(x)$ means everyone in the world is a student and is smart.

- Existential quantification $\exists$
 - $(\exists x)P(x)$ means that P holds for some value(s) of x in the domain associated with that variable.
 - For example.,
 - $(\exists x)\text{mammal}(x) \wedge \text{lays-eggs}(x)$
 - $(\exists x)\text{taller}(x, \text{Ngozi})$
 - $(\exists x)\text{Student}(x) \wedge \text{taller}(x, \text{Emeka})$
 - Existential quantifiers usually used with “ $\wedge$ (and)” to specify a list of properties about an individual.
 - $(\exists x)\text{student}(x) \wedge \text{smart}(x)$ (there is a student who is smart.)
 - A common mistake is to represent this English sentence as the FOL sentence:
 - $(\exists x)\text{student}(x) \Rightarrow \text{smart}(x)$
 - It also holds if no student exists in the domain because $\text{student}(x) \Rightarrow \text{smart}(x)$ holds for any individual who is not a student.
 - Often associated with English words “someone”, “sometimes”, etc.

Scopes of Quantifiers

- Each quantified variable has its scope
 - $(\forall x)[human(x) \Rightarrow (\exists y)[human(y) \wedge father(y, x)]]$
 - All occurrences of x within the scope of the quantified x refer to the same thing.
 - Use different variables for different things
- Switching the order of universal quantifiers does not change the meaning:
 - $(\forall x)(\forall y)P(x, y) \Leftrightarrow (\forall y)(\forall x)P(x, y)$, can write as $(\forall x, y)P(x, y)$
- Similarly, you can switch the order of existential quantifiers.
 - $(\exists x)(\exists y)P(x, y) \Leftrightarrow (\exists y)(\exists x)P(x, y)$
- Switching the order of universals and existential does change meaning:
 - Everyone likes someone: $(\forall x)(\exists y)likes(x, y)$
 - Someone is liked by everyone: $(\exists y)(\forall x)likes(x, y)$

Sentences are built from terms and atoms

- A **term** (denoting an individual in the world) is a constant symbol, a variable symbol, or a function of terms.
- An **atom** (atomic sentence) is a predicate $P(x[1], \dots, x[n])$
 - Ground atom: all terms in its arguments are ground terms (do not involve variables)
 - A ground atom has value true or false (like a proposition in PL)
- A **literal** is either an atom or a negation of an atom
- A **sentence** is an atom, or,
 - $\neg P, P \vee Q, P \wedge Q, P \Rightarrow Q, P \Leftrightarrow Q, (P)$ where P and Q are sentences
 - If P is a sentence and x is a variable, then $(\forall x)P$ and $(\exists x)P$ are sentences
- A **well-formed formula** (wff) is a sentence containing no "free" variables. i.e., all variables are "bound" by universal or existential quantifiers. $(\forall x)P(x, y)$ has x bound as a universally quantified variable, but y is free.

A BNF for FOL Sentences

```
S := <Sentence>;  
<Sentence> := <AtomicSentence> |  
    <Sentence><Connective><Sentence> |  
    <Quantifier><Variable>,... <Sentence> |  
     $\neg$  <Sentence> | "(" <Sentence> ")";  
<AtomicSentence> := <Predicate> "(" <Term>, ... ")" |  
    <Term> "=" <Term>;  
<Term> := <Function> "(" <Term>, ... ")" |  
    <Constant> | <Variable>;  
<Connective> :=  $\wedge$  |  $\vee$  |  $\Rightarrow$  |  $\Leftrightarrow$ ;  
<Quantifier> :=  $\forall$  |  $\exists$ ;  
<Constant> := "A" | "X1" | "John" | ...;  
<Variable> := "a" | "x" | "s" | ...;  
<Predicate> := "Before" | "HasColor" | "Raining" | ... ;  
<Function> := "Mother" | "LeftLegOf" | ... ;  
<Literal> := <AtomicSentence> |  $\neg$  <AtomicSentence>
```

Translating English to FOL

- Every gardener likes the sun.

$$(\forall x)gardener(x) \Rightarrow likes(x, Sun)$$

- Not Every gardener likes the sun.

$$\sim ((\forall x)gardener(x) \Rightarrow likes(x, Sun))$$

- You can fool some of the people all of the time.

$$(\exists x)(\forall t)person(x) \wedge time(t) \Rightarrow can - be - fooled(x, t)$$

- You can fool all of the people some of the time.

$$(\forall x)(\exists t)person(x) \wedge time(t) \Rightarrow can - be - fooled(x, t)$$

(the time people are fooled may be different)

- You can fool all of the people at some time.

$$(\exists t)(\forall x)person(x) \wedge time(t) \Rightarrow can - be - fooled(x, t)$$

(all people are fooled at the same time)

Translating English to FOL

- You can not fool all of the people all of the time.

$$\neg((\forall x)(\forall t)person(x) \wedge time(t) \wedge can - be - fooled(x, t))$$

- Everyone is younger than his father

$$(\forall x)person(x) \Rightarrow younger(x, father(x))$$

- All purple mushrooms are poisonous.

$$(\forall x)(mushroom(x) \wedge purple(x)) \Rightarrow poisonous(x)$$

- No purple mushroom is poisonous.

$$\neg(\exists x)purple(x) \wedge mushroom(x) \wedge poisonous(x)$$

$$(\forall x)(mushroom(x) \wedge purple(x)) \Rightarrow \neg poisonous(x)$$

- X is above Y if X is directly on top of Y or there is a pile of one or more other objects directly on top of one another starting with X and ending with Y.

$$(\forall x)(\forall y)above(x, y) \Leftrightarrow (on(x, y) \vee (\exists z)(on(x, z) \wedge above(z, y)))$$

Semantics of FOL

- Domain M : the set of all objects in the world (of interest)
- Interpretation I : includes
 - Assign each constant to an object in M
 - Define each function of n arguments as a mapping $M^n \rightarrow M$
 - Define each predicate of n arguments as a mapping $M^n \rightarrow \{T, F\}$
 - Therefore, every ground predicate with any instantiation will have a truth value
 - In general there are infinite number of interpretations because $|M|$ is infinite
- Define logical connectives: $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$ as in PL
- Define semantics of $(\forall x)$ and $(\exists x)$
 - $(\forall x)P(x)$ is true iff $P(x)$ is true under all interpretations
 - $(\exists x)P(x)$ is true iff $P(x)$ is true under some interpretation

- **Model:** an interpretation of a set of sentences such that every sentence is True
- A **sentence** is
 - satisfiable if it is true under some interpretation
 - valid if it is true under all possible interpretations
 - inconsistent if there does not exist any interpretation under which the sentence is true
- logical consequence: $S \models X$ if all models of S are also models of X

Axioms, Definitions, and theorems

- **Axioms** are facts and rules which are known (or assumed) to be true facts and concepts about a domain.
 - Mathematicians don't want any unnecessary (dependent) axioms – ones that can be derived from other axioms.
 - Dependent axioms can make reasoning faster, however.
 - Choosing a good set of axioms for a domain is a kind of design problem.
- A **definition** of a predicate is of the form " $P(x) \Leftrightarrow S(x)$ " (define $P(x)$ by $S(x)$) and can be decomposed into two parts
 - Necessary description: " $P(x) \Rightarrow S(x)$ " (only if)
 - Sufficient description " $P(x) \Leftarrow S(x)$ " (if)
 - Some concepts don't have complete definitions (e.g. $\text{person}(x)$)
- A **theorem** S is a sentence that logically follows the axiom set A , i.e. $A \models S$.

Higher-order logic (HOL)

- FOL only allows to quantify over variables, and variables can only range over objects.
- HOL allows us to quantify over relations
- Example: (quantify over functions)
 - “two functions are equal iff they produce the same value for all arguments”
 - $\forall f \forall g (f = g) \Leftrightarrow (\forall x f(x) = g(x))$
- Example: (quantify over predicates)
 - $\forall r \text{transitive}(r) \Rightarrow (\forall x y z) r(x, y) \wedge r(y, z) \Rightarrow r(x, z)$
- More expressive, but undecidable.

More Definitions

- A **definition** of $P(x)$ by $S(x)$, denoted $(\forall x)P(x) \Leftrightarrow S(x)$, can be decomposed into two parts
 - Necessary description: " $P(x) \Rightarrow S(x)$ " (only if, for $P(x)$ being true, $S(x)$ is necessarily true)
 - Sufficient description " $P(x) \Leftarrow S(x)$ " (if, $S(x)$ being true is sufficient to make $P(x)$ true)
- Examples: define $\text{father}(x, y)$ by $\text{parent}(x, y)$ and $\text{male}(x)$
 - $\text{parent}(x, y)$ is a necessary (but not sufficient) description of $\text{father}(x, y)$ $\text{father}(x, y) \Rightarrow \text{parent}(x, y), \text{parent}(x, y) \Rightarrow \text{father}(x, y)$
 - $\text{parent}(x, y) \wedge \text{male}(x)$ is a necessary and sufficient description of $\text{father}(x, y)$
$$\text{parent}(x, y) \wedge \text{male}(x) \Leftrightarrow \text{father}(x, y)$$
 - $\text{parent}(x, y) \wedge \text{male}(x) \wedge \text{age}(x, 35)$ is a sufficient (but not necessary) description of $\text{father}(x, y)$ because

$$\text{father}(x, y) \Rightarrow \text{parent}(x, y) \wedge \text{male}(x) \wedge \text{age}(x, 35)$$

Theorem

1. $\neg(\forall x P(x)) \equiv \exists x \neg P(x)$
2. $\neg(\exists x P(x)) \equiv \forall x \neg P(x)$
3. $\exists x(P(x) \Rightarrow Q(x)) \equiv \forall x P(x) \Rightarrow \exists x Q(x)$
4. $\exists x(P(x) \vee Q(x)) \equiv \exists x P(x) \vee \exists x Q(x)$
5. $\forall x(P(x) \wedge Q(x)) \equiv \forall x P(x) \wedge \forall x Q(x)$
6. $(\forall x P(x)) \vee (\forall x Q(x)) \equiv \forall x(P(x) \vee Q(x))$
7. $\exists x(P(x) \wedge Q(x)) \equiv \exists x P(x) \wedge \exists x Q(x)$

Inference in First-Order Logic

Inference in First-Order Logic

- Inference rules for PL apply to FOL as well (Modus Ponens – (modus ponendo ponens) “the way that affirms by affirming” or “affirming the consequent”, And-Introduction, And-Elimination, etc.)
- New (sound) inference rules for use with quantifiers:
 - Universal Elimination
 - Existential Introduction
 - Existential Elimination
 - Generalized Modus Ponens (GMP)
- Resolution
 - Clause form (CNF in FOL)
 - Unification (consistent variable substitution)
 - Refutation resolution (proof by contradiction)

- **Universal Elimination** $(\forall x)P(x) \vdash P(c)$.
 - If $(\forall x)P(x)$ is true, then $P(c)$ is true for any constant c in the domain of x , i.e., $(\forall x)P(x) \models P(c)$.
 - Replace all occurrences of x in the scope of $\forall x$ by the same ground term (a constant or a ground function).
 - Example: $(\forall x)eats(Chichi, x) \vdash eats(Chichi, IceCream)$
- **Existential Introduction** $P(c) \vdash (\exists x)P(x)$
 - If $P(c)$ is true, so is $(\exists x)P(x)$, i.e., $P(c) \models (\exists x)P(x)$
 - Replace all instances of the given constant symbol by the same new variable symbol.
 - Example $eats(Chichi, IceCream) \vdash (\exists x) eats(Chichi, x)$
- **Existential Elimination**
 - From $(\exists x)P(x)$ infer $P(c)$, i.e., $(\exists x)P(x) \models P(c)$, where c is a new constant symbol,
 - All we know is there must be some constant that makes this true, so we can introduce a brand new one to stand in for that constant, even though we don't know exactly what that constant refers to.
 - Example: $(\exists x)eats(Chichi, x) \models eats(Chichi, Stuff)$

Skolemization

- Things become more complicated when there are universal quantifiers

$$(\forall x)(\exists y)eats(x, y) \models (\forall x)eats(x, Stuff)???$$

$$(\forall x)(\exists y)eats(x, y) \models eats(Chichi, Stuff)???$$

- Introduce a new function $food_sk(x)$ to stand for $\exists y$ because that y depends on x

$$(\forall x)(\exists y)eats(x, y) \vdash (x)eats(x, food_sk(x))$$

$$(\forall x)(\exists y)eats(x, y) \vdash eats(Chichi, food_sk(Chichi))$$

- What exactly the function $food_sk(.)$ does is unknown, except that it takes x as its argument
- The process of existential elimination is called “Skolemization”, and the new, unique constants (e.g., $Stuff$) and functions (e.g., $food_sk(.)$) are called skolem constants and skolem functions, respectively.

Generalized Modus Ponens

- Combines And-Introduction, Universal-Elimination, and Modus Ponens
- Ex: $P(c), Q(c), (\forall x)(P(x) \wedge Q(x)) \Rightarrow R(x) \vdash R(c)$
 $P(c), Q(c) \vdash P(c) \wedge Q(c)$ (by and-introduction)
 $(\forall x)(P(x) \wedge Q(x)) \Rightarrow R(x)$
 $\vdash (P(c) \wedge Q(c)) \Rightarrow R(c)$ (by universal-elimination)
 $P(c) \wedge Q(c), (P(c) \wedge Q(c)) \Rightarrow R(c) \vdash R(c)$ (by modus ponens)
- All occurrences of a quantified variable must be instantiated to the same constant.

$$P(a), Q(c), (\forall x)(P(x) \wedge Q(x)) \Rightarrow R(x) \vdash R(c)$$

because all occurrences of x must either be instantiated to a or c which makes the modus ponens rule not applicable.

Resolution for FOL

- Resolution rule operates on two clauses
 - A clause is a **disjunction** of literals (without explicit quantifiers)
 - Relationship between clauses in KB is **conjunction**
- Resolution Rule for FOL:
 - clause $C_1 : (l_1, l_2, \dots, l_i, \dots, l_n)$ and clause $C_2 : (l_1, l_2, \dots, l_j, \dots, l_m)$
 - if l_i and l_j are two opposite literals (e.g., P and $\neg P$) and their argument lists can be made the same (unified) by a set of variable bindings $q = \{x_1/y_1, \dots, x_k/y_k\}$ where $x_1, \dots, x_k$ are variables and $y_1, \dots, y_k$ are terms, then derive a new clause (called resolvent)

$$\text{subst}((l_1, l_2, \dots, l_n, l_1, l_2, \dots, l_m), q)$$

where function $\text{subst}(\text{expression}, q)$ returns a new expression by applying all variable bindings in q to the original expression

We need answers to the following questions

- How to convert FOL sentences to clause form (especially how to remove quantifiers)
- How to unify two argument lists, i.e., how to find their most general unifier (mgu) θ
- How to determine which two clauses in KB should be resolved next (among all resolvable pairs of clauses) and how to determine a proof is completed

Converting FOL sentences to clause form

- Clauses are quantifier free CNF of FOL sentences
- Basic ideas
 - How to handle quantifiers
 - Careful on quantifiers with preceding negations (explicit or implicit)

$\neg\forall xP(x)$ is really $\exists x\neg P(x)$

$$\begin{aligned}(\forall xP(x)) \Rightarrow (\forall yQ(y)) &\equiv \neg(\forall xP(x)) \vee (\forall yQ(y)) \\ &\equiv \exists x\neg P(x) \vee \forall yQ(y)\end{aligned}$$

- Eliminate true existential quantifier by Skolemization
 - For true universally quantified variables, treat them as such without quantifiers
- How to convert to CNF (similar to PL but need to work with quantifiers)

Conversion Procedure

- step 1 : remove all “ $\Rightarrow$ ” and “ $\Leftrightarrow$ ” operators (using $P \Rightarrow Q \equiv \neg P \vee Q$ and $P \Leftrightarrow Q \equiv P \Rightarrow Q \wedge Q \Rightarrow P$)
- step 2 : move all negation signs to individual predicates (using de Morgan's law)
- step 3 : remove all existential quantifiers $\exists y$
- case 1 : y is not in the scope of any universally quantified variable, then replace all occurrences of y by a **skolem constant**
 - case 2 : if y is in scope of universally quantified variables $x_1, \dots, x_i$, then replace all occurrences of y by a **skolem function**
- step 4 : remove all universal quantifiers x (with the understanding that all remaining variables are universally quantified)
- step 5 : convert the sentence into CNF (using distribution law, etc)
- step 6 : use parenthesis to separate all disjunctions, then drop all $\vee$'s and $\wedge$'s

Putting Axioms in Clause Form – an illustration

- Consider the following axiom:
 - First, a brick is on something that is not a pyramid; second, there is nothing that a brick is on and that is on the brick as well; and third, there is nothing that is not a brick and also is the same thing as the brick:

$$\begin{aligned} &\forall x[Brick(x) \Rightarrow (\exists y[On(x, y) \wedge \neg Pyramid(y)] \\ &\quad \wedge \neg \exists y[On(x, y) \wedge On(y, x)] \\ &\quad \wedge \forall y[\neg Brick(y) \Rightarrow \neg Equal(x, y)])] \end{aligned}$$

- Clearly, the above axiom is not in clause form
- Therefore, it cannot be used to produce resolvents
- The axiom has to be transformed into one or more equivalent axioms in clause form
- Below are the steps needed to convert the axiom into clause form

Step1: Eliminate Implications

- To perform this step, simply substitute $\neg A \vee B$ for $A \Rightarrow B$. For the example, substitutions are required, producing:

$$\begin{aligned} & \forall x [\neg Brick(x) \vee (\exists y [On(x, y) \wedge \neg Pyramid(y)] \\ & \wedge \neg \exists y [On(x, y) \wedge On(y, x)] \\ & \wedge \forall y [\neg (\neg Brick(y)) \vee \neg Equal(x, y)]]] \end{aligned}$$

Step 2: move negations down to the atomic formulas

- Performing this step on the example produces

$$\begin{aligned} & \forall x [\neg Brick(x) \vee (\exists y [On(x, y) \wedge \neg Pyramid(y)] \\ & \wedge \forall y [\neg On(x, y) \vee \neg On(y, x)] \\ & \wedge \forall y [Brick(y) \vee \neg Equal(x, y)])] \end{aligned}$$

Step 3: Eliminate existential quantifiers

- To perform this step, we introduce **Skolem functions**.

$$\begin{aligned} & \forall x [\neg \text{Brick}(x) \vee ([\text{On}(x, \text{sk}(x)) \wedge \neg \text{Pyramid}(\text{sk}(x))]) \\ & \wedge \forall y [\neg \text{On}(x, y) \vee \neg \text{On}(y, x)] \\ & \wedge \forall y [\text{Brick}(y) \vee \neg \text{Equal}(x, y)]] \end{aligned}$$

Step 4: Rename variables

- Rename variables, as necessary, so no two variables are the same.

$$\begin{aligned} & \forall x [\neg \text{Brick}(x) \vee ([\text{On}(x, \text{sk}(x)) \wedge \neg \text{Pyramid}(\text{sk}(x))]) \\ & \wedge \forall y [\neg \text{On}(x, y) \vee \neg \text{On}(y, x)] \\ & \wedge \forall z [\text{Brick}(z) \vee \neg \text{Equal}(x, z)]] \end{aligned}$$

Step 5: Move the universal quantifiers to the left

- This produces:

$$\begin{aligned} & \forall x \forall y \forall z [\neg Brick(x) \vee ([On(x, sk(x)) \wedge \neg Pyramid(sk(x))] \\ & \wedge [\neg On(x, y) \vee \neg On(y, x)] \\ & \wedge [Brick(z) \vee \neg Equal(x, z)])] \end{aligned}$$

Step 6: Move the disjunctions down to the literals

- Performing this in two steps yields:

-

$$\begin{aligned} & \forall x \forall y \forall z [\neg Brick(x) \vee ([On(x, sk(x)) \wedge \neg Pyramid(sk(x))]) \\ & \wedge [\neg Brick(x) \vee \neg On(x, y) \vee \neg On(y, x)] \\ & \wedge [\neg Brick(x) \vee Brick(z) \vee \neg Equal(x, z)]]] \end{aligned}$$

-

$$\begin{aligned} & \forall x \forall y \forall z [(\neg Brick(x) \vee On(x, sk(x))) \\ & \wedge (\neg Brick(x) \vee \neg Pyramid(sk(x))) \\ & \wedge [\neg Brick(x) \vee \neg On(x, y) \vee \neg On(y, x)] \\ & \wedge [\neg Brick(x) \vee Brick(z) \vee \neg Equal(x, z)]]] \end{aligned}$$

Step 7: Eliminate the conjunctions

- This produces:

$$\forall x[\neg Brick(x) \vee On(x, sk(x))]$$

$$\forall x[\neg Brick(x) \vee \neg Pyramid(sk(x))]$$

$$\forall x \forall y[\neg Brick(x) \vee \neg On(x, y) \vee \neg On(y, x)]$$

$$\forall x \forall z[\neg Brick(x) \vee Brick(z) \vee \neg Equal(x, z)]$$

Step 8: Rename all variables

- Rename all variables, as necessary, so that no two variables are the same.

$$\forall x[\neg Brick(x) \vee On(x, sk(x))]$$

$$\forall u[\neg Brick(u) \vee \neg Pyramid(sk(u))]$$

$$\forall v \forall y[\neg Brick(v) \vee \neg On(v, y) \vee \neg On(y, v)]$$

$$\forall w \forall z[\neg Brick(w) \vee Brick(z) \vee \neg Equal(w, z)]$$

Step 9: Eliminate the universal quantifiers

- Produces: The result is now in clause form.

$$\neg Brick(x) \vee On(x, sk(x))$$

$$\neg Brick(u) \vee \neg Pyramid(sk(u))$$

$$\neg Brick(v) \vee \neg On(v, y) \vee \neg On(y, v)$$

$$\neg Brick(w) \vee Brick(z) \vee \neg Equal(w, z)$$

To put axioms in clause form: Summary of the procedure

- Eliminate the implications
- Move the negations down to the atomic formulas
- Eliminate existential quantifiers
- Rename the variables, if necessary
- Move the universal quantifiers to the left
- Move the disjunctions to the literals
- Eliminate the conjunctions
- Rename the variables, if necessary
- Eliminate the universal quantifiers
- use parenthesis to separate all disjunctions, then drop all $\vee$'s and $\wedge$'s

Example

Convert the following sentence to clause form: $\forall x(P(x) \wedge Q(x) \Rightarrow R(x))$.

Solution

$$\forall x \neg(P(x) \wedge Q(x)) \vee R(x) \quad (\text{by step 1})$$

$$\forall x \neg P(x) \vee \neg Q(x) \vee R(x) \quad (\text{by step 2})$$

$$\neg P(x) \vee \neg Q(x) \vee R(x) \quad (\text{by step 5})$$

$$(\neg P(x), \neg Q(x), R(x)) \quad (\text{by step 10})$$

Example

Convert the following sentence to clause form:

$\forall x[\text{person}(x) \Rightarrow \exists y(\text{person}(y) \wedge \text{father}(y, x))]$. **Solution**

$\forall x[\neg \text{person}(x) \vee \exists y(\text{person}(y) \wedge \text{father}(y, x))]$ (by step 1)

$\forall x[\neg \text{person}(x) \vee (\text{person}(f_sk(x)) \wedge \text{father}(f_sk(x), x))]$ (by step 3)

$[\neg \text{person}(x) \vee (\text{person}(f_sk(x)) \wedge \text{father}(f_sk(x), x))]$ (by step 5)

$(\neg \text{person}(x) \vee \text{person}(f_sk(x))) \wedge (\neg \text{person}(x) \vee \text{father}(f_sk(x), x))$ (by step 6)

$(\neg \text{person}(x), \text{person}(f_sk(x))), (\neg \text{person}(x), \text{father}(f_sk(x), x))$ (by step 7/10)

Unification of two clauses

Unification of Two Clauses

- Basic idea: $\forall x P(x) \Rightarrow Q(x), P(a) \vdash Q(a)$

$(\neg P(x) \quad Q(x))$

$(P(a))$

$Q(a)$

$\{x/a\}$ a substitution in which
variable x is bound to a

- The goal is to find a set of variable bindings so that the argument lists of two opposite literals (in two clauses) can be made the same.
- Only variables can be bound to other things.
 - a and b cannot be unified (different constants in general refer to different objects)
 - a and $f(x)$ cannot be unified (unless the inverse function of f is known, which is not the case for general functions in FOL)
 - $f(x)$ and $g(y)$ cannot be unified (function symbols f and g in general refer to different functions and their exact definitions are different in different interpretations)

- Cannot bind variable x to y if x appears anywhere in y
 - Try to unify x and $f(x)$. If we bind x to $f(x)$ and apply the binding to both x and $f(x)$, we get $f(x)$ and $f(f(x))$ which are still not the same (and will never be made the same no matter how many times the binding is applied)
- Otherwise, bind variable x to y , written as x/y (this guarantees to find the most general unifier, or **mg**u)
 - Suppose both x and y are variables, then they can be made the same by binding both of them to any constant c or any function $f(\cdot)$. Such bindings are less general and impose unnecessary restriction on x and y .
- To unify two terms of the same function symbol, unify their argument lists (**unification is recursive**) Ex: to unify $f(x)$ and $f(g(b))$, we need to unify x and $g(b)$

- When the argument lists contain multiple terms, unify each pair of terms Ex. To unify $(x, f(x), \dots)$ $(a, y, \dots)$
 1. unify x and a ($q = \{x/a\}$)
 2. apply q to the remaining terms in both lists, resulting $(f(a), \dots)$ and $(y, \dots)$
 3. unify $f(a)$ and y with binding $y/f(a)$
 4. apply the new binding $y/f(a)$ to q
 5. add $y/f(a)$ to new q

Unification examples

- $\text{parents}(x, \text{father}(x), \text{mother}(\text{Chike}))$ and $\text{parents}(\text{Bill}, \text{father}(\text{Chike}), y)$
 - unify x and Chike : $q = \{x/\text{Chike}\}$
 - unify $\text{father}(\text{Chike})$ and $\text{father}(\text{Chike})$: $q = \{x/\text{Chike}\}$
 - unify $\text{mother}(\text{Chike})$ and y : $q = \{x/\text{Chike}, y/\text{mother}(\text{Chike})\}$
- $\text{parents}(x, \text{father}(x), \text{mother}(\text{Chike}))$ and $\text{parents}(\text{Chike}, \text{father}(y), z)$
 - unify x and Chike : $q = \{x/\text{Chike}\}$
 - unify $\text{father}(\text{Chike})$ and $\text{father}(y)$: $q = \{x/\text{Chike}, y/\text{Chike}\}$
 - unify $\text{mother}(\text{Chike})$ and z : $q = \{x/\text{Chike}, y/\text{Chike}, z/\text{mother}(\text{Chike})\}$
- $\text{parents}(x, \text{father}(x), \text{mother}(\text{Jane}))$ and $\text{parents}(\text{Chike}, \text{father}(y), \text{mother}(y))$
 - unify x and Chike : $q = \{x/\text{Chike}\}$
 - unify $\text{father}(\text{Chike})$ and $\text{father}(y)$: $q = \{x/\text{Chike}, y/\text{Chike}\}$
 - unify $\text{mother}(\text{Jane})$ and $\text{mother}(\text{Chike})$: Failure because Jane and Bill are different constants

More unification examples

- $P(x, g(x), h(b))$ and $P(f(u, a), v, u)$
 - unify x and $f(u, a)$: $q = \{x/f(u, a)\}$;
remaining lists: $(g(f(u, a)), h(b))$ and (v, u)
 - unify $g(f(u, a))$ and v : $q = \{x/f(u, a), v/g(f(u, a))\}$;
remaining lists: $(h(b))$ and (u)
 - unify $h(b)$ and u : $q = \{x/f(h(b), a), v/g(f(h(b), a)), u/h(b)\}$;
- $P(f(x, a), g(x, b))$ and $P(y, g(y, b))$
 - unify $f(x, a)$ and y : $q = \{y/f(x, a)\}$
remaining lists: $(g(x, b))$ and $(g(f(x, a), b))$
 - unify x and $f(x, a)$: failure because x is in $f(x, a)$

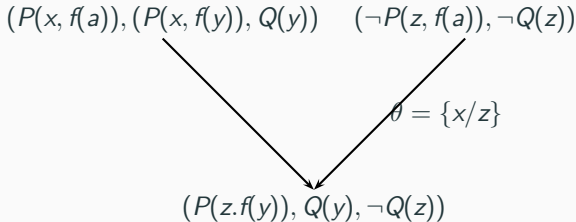
Unification Algorithm

```
procedure unify(p, q,  $\theta$ ) /* p and q are two lists of terms and  $|p| = |q|$  */
  if p = empty then return q; /* success */
  let r = first(p) and s = first(q);
  if r = s then return unify(rest(p), rest(q),  $\theta$ );
  if r is a variable then temp = unify-var(r, s);
  else if s is a variable then temp = unify-var(s, r);
    else if both r and s are functions of the same function name then
      temp = unify(arglist(r), arglist(s), empty);
    else return "failure";
  if temp = "failure" then return "failure"; /* p and q are not unifiable */
  else  $\theta$  = subst( $\theta$ , temp) temp; /* apply tmp to old q then insert it into  $\theta$  */
  return unify(subst(rest(p), tmp), subst(rest(q), tmp),  $\theta$ );
end{unify}

procedure unify-var(x, y)
  if x appears anywhere in y then return "failure";
  else return (x/y)
end{unify-var}
```

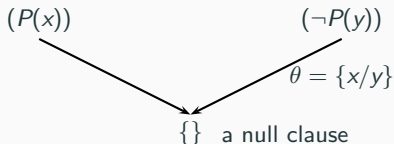
Resolution in FOL

- Convert all sentences in KB (axioms, definitions, and known facts) and the goal sentence (the theorem to be proved) to clause form
- Two clauses C_1 and C_2 can be resolved if and only if r in C_1 and s in C_2 are two opposite literals, and their argument list $arglist_r$ and $arglist_s$ are unifiable with $mgu = \theta$.
- Then derive the resolvent sentence: $subst((C_1\{r\}, C_2\{s\}), \theta)$
(substitution is applied to all literals in C_1 and C_2 , but not to any other clauses)



Resolution Refutation: a better proof strategy

- Given a consistent set of axioms KB and goal sentence Q , show that $KB \models Q$.
- **Proof by contradiction:** Add $\neg Q$ to KB and try to prove false.
because $(KB \models Q) \Leftrightarrow (KB \wedge \neg Q \models \text{False})$, or $KB \wedge \neg Q$ is inconsistent)
- How to represent “false” in clause form
 - $P(x) \wedge \neg P(y)$ is inconsistent
 - Convert them to clause form then apply resolution



- A null clause represents false (inconsistence/contradiction)
- $KB \models Q$ if we can derive a null clause from $KB \wedge \neg Q$ by resolution

Refutation Resolution Procedure

```
procedure resolution(KB, Q)
  /* KB is a set of consistent, true FOL sentences, Q is a goal sentence.
  It returns success if  $KB \models Q$ , and failure otherwise */
  KB = clause(union(KB, { $\neg Q$ })) /* convert KB and Q to clause form */
  while null clause is not in KB do
    pick 2 sentences,  $S_1$  and  $S_2$ , in KB that contain a pair of opposite
      literals whose argument lists are unifiable
    if none can be found then return "failure"
    resolvent = resolution-rule( $S_1, S_2$ )
    KB = union(KB, {resolvent})
  return "success "
end{resolution}
```

References

- [1] Patrick H Winston, Artificial Intelligence, Addison Wesley.
- [2] George F Luger, Artificial Intelligence, Addison Wesley.